

Production Quality Assurance

Audit Report: StackPilot AI

Product Name: StackPilot AI **Product Type:** Enterprise AI-powered Project Management, CRM, Finance, HR, SEO & Operations Platform **Target**

Environments:

- **Frontend:** `https://stackpilot-ai-seven.vercel.app` (Vite, React, TypeScript, Tailwind CSS)
- **Backend API:** `https://stackpilot-ai-clp6.onrender.com` (Node.js, Express, MongoDB, JWT) **Lead QA Automation Engineer Audit Date:** July 23, 2026 **Audit Scope:** 10 Phases (Functional, UI/UX, API, Security OWASP Top 10, Performance, Edge Cases, Cross-Browser, Accessibility WCAG 2.1 AA, Automation Suites, Production Readiness Audit). ---

Executive Summary & Key Findings

A thorough production-level Quality Assurance audit was conducted across the entire StackPilot AI platform. While the system presents a high-level UI ("White & Light Green" design system, responsive dashboards, and interactive mock/live API fallback handling), critical vulnerabilities and bugs were discovered during negative testing, security scanning, and payload injection tests.

CAUTION: CRITICAL SECURITY RISK: NoSQL Injection Vulnerability

> The backend `login` controller passes user inputs directly into Mongoose queries without sanitization. An attacker can authenticate as the Super Admin account without knowing the email or password by submitting a MongoDB operator payload (`{ "email": { "$gt": "" }, "password": "password123" }`).

WARNING: HIGH SECURITY RISK: Predictable Hardcoded JWT Secret

Fallback

> In `middleware/auth.ts` and `controllers/index.ts`, `JWT_SECRET` defaults to `'stackpilot_secret_key_12345'`. Attackers can forge valid administrative JWT tokens offline using HS256 algorithm signing. ---

Phase Audit Defect Taxonomy

Defect #1: NoSQL Injection Authentication Bypass (CRITICAL)

- **Priority:** Critical
- **Module:** Authentication & Security
- **Description:** Direct query object injection allows unauthenticated users to bypass login and acquire valid JWT admin credentials.
- **Steps to reproduce:** 1. Send `POST /api/auth/login` request. 2. Set HTTP Body: `{"email": {"$gt": ""}, "password": "password123"}`. 3. Observe server response.
- **Expected behaviour:** Server should reject non-string input types for `email` and return `400 Bad Request`.
- **Actual behaviour:** Server returns `HTTP 200 OK` with a valid JWT token signed for `admin@stackpilot.ai`.
- **Screenshot / Code reference:** [apps/api/src/controllers/index.ts:63-65](#)
- **Suggested fix:** Validate that `req.body.email` and `req.body.password` are primitive strings before passing them to Mongoose query methods.
- **Developer recommendation:** Install `express-mongo-sanitize` middleware or enforce `Zod / Joi` payload validation schemas on all public routes. ---

Defect #2: Hardcoded Fallback JWT Secret (CRITICAL)

- **Priority:** Critical
- **Module:** Authentication & Security

- **Description:** Hardcoded fallback key `stackpilot_secret_key_12345` is stored in source code.
- **Steps to reproduce:** 1. Inspect `middleware/auth.ts` line 4 and `controllers/index.ts` line 7. 2. Generate a local JWT token using secret `stackpilot_secret_key_12345` with payload `{ "id": "admin_id", "role": "Super Admin" }`. 3. Send request to `GET /api/projects` with header `Authorization: Bearer` .
- **Expected behaviour:** Secret key should only be loaded from environment variables (`process.env.JWT_SECRET`), and server should refuse to boot if `JWT_SECRET` is missing in production.
- **Actual behaviour:** Server defaults to publicly exposed secret `'stackpilot_secret_key_12345'`, allowing offline token forgery.
- **Screenshot / Code reference:** [apps/api/src/middleware/auth.ts:4](#)
- **Suggested fix:** Throw a fatal error on server startup if `process.env.JWT_SECRET` is undefined or less than 32 characters long.
- **Developer recommendation:** Enforce environment variable check in `server.ts` before `app.listen()`. ---

Defect #3: Missing Role-Based Access Control (RBAC) Enforcement on Backend (HIGH)

- **Priority:** High
- **Module:** Security & Authorization
- **Description:** `requireRoles` middleware exists in `middleware/auth.ts` but is NEVER attached to API routes in `server.ts`.
- **Steps to reproduce:** 1. Log in as a user with role `Client` or `Developer`. 2. Send `POST /api/finance/invoices` Or `POST /api/projects`.
- **Expected behaviour:** Server should reject non-Admin / non-Finance users with `403 Forbidden: You do not have permissions for this action`.
- **Actual behaviour:** Backend executes creation logic for any authenticated user regardless of assigned role.

- **Screenshot / Code reference:** [apps/api/src/server.ts:34-57](#)
- **Suggested fix:** Apply `requireRoles(['Super Admin', 'Admin', 'Finance'])` to administrative endpoints in `server.ts`.
- **Developer recommendation:** Update route definitions in Express server to enforce strict authorization middleware. ---

Defect #4: Unhandled Mongoose CastError on Invalid ObjectIDs (HIGH)

- **Priority:** High
- **Module:** API Error Handling & Tasks Module
- **Description:** Sending non-Hex 24-character strings as document IDs triggers an unhandled `CastError`, returning `500 Internal Server Error` with stack trace leak.
- **Steps to reproduce:** 1. Send `PUT /api/tasks/invalid-mongo-id-123` with valid JWT token.
- **Expected behaviour:** Server returns clean `400 Bad Request ({ "error": "Invalid task ID format" })`.
- **Actual behaviour:** Server returns `500 Internal Server Error` exposing raw Mongoose `CastError` exception details.
- **Screenshot / Code reference:** [apps/api/src/controllers/index.ts:217-225](#)
- **Suggested fix:** Wrap `findByIdAndUpdate` calls or add standard Express global error handler checking `err.name === 'CastError'`.
- **Developer recommendation:** Implement centralized error-handling middleware (`app.use((err, req, res, next) => ...)`). ---

Defect #5: Invisible Heading Text on Login Page (HIGH)

- **Priority:** High
- **Module:** UI / UX & Authentication
- **Description:** The main heading on the Login page uses CSS class `text-white` on a white background container (`bg-white`), rendering text invisible

to users.

- **Steps to reproduce:** 1. Navigate to <https://stackpilot-ai-seven.vercel.app/login>. 2. Observe heading "Welcome back to StackPilot".
- **Expected behaviour:** Heading text should use high-contrast dark color (`text-slate-900` or `#111827`).
- **Actual behaviour:** Text color is `#FFFFFF`, resulting in 1:1 contrast ratio (invisible text).
- **Screenshot / Code reference:** [apps/web/src/features/auth/Login.tsx:58](#)
- **Suggested fix:** Replace `text-white` with `text-slate-900` on line 58 of `Login.tsx`.
- **Developer recommendation:** Run automated WCAG contrast checks during CI build pipeline. ---

Defect #6: Missing HTTP Security Headers & Rate Limiting (HIGH)

- **Priority:** High
- **Module:** Security & Network
- **Description:** Production Express API server lacks essential HTTP security headers (HSTS, CSP, X-Frame-Options, X-Content-Type-Options) and rate-limiting controls.
- **Steps to reproduce:** 1. Send HTTP GET request to <https://stackpilot-ai-clp6.onrender.com/api/seo/reports>. 2. Inspect response headers.
- **Expected behaviour:** Headers must include `Strict-Transport-Security`, `X-Content-Type-Options: nosniff`, `X-Frame-Options: DENY`, and `X-RateLimit-*`.
- **Actual behaviour:** None of these headers are returned by Express.
- **Screenshot / Code reference:** [apps/api/src/server.ts:18-24](#)
- **Suggested fix:** Register `helmet()` and `express-rate-limit` middleware in `Express app.use()`.
- **Developer recommendation:** Standardize API gateway security headers. ---

Defect #7: Frontend AuthContext Login Parameter Omission (MEDIUM)

- **Priority:** Medium
- **Module:** Frontend Auth & State
- **Description:** `AuthContext.tsx` defines `login: (email: string) => Promise` omitting password parameter.
- **Steps to reproduce:** 1. Inspect `apps/web/src/context/AuthContext.tsx` line 53. 2. Call `login(email)`.
- **Expected behaviour:** `login` method should accept both `email` and `password` parameters.
- **Actual behaviour:** `password` is ignored on frontend auth state helper, defaulting to fixed string when calling backend API in `api.ts`.
- **Screenshot / Code reference:** [apps/web/src/context/AuthContext.tsx:53](#)
- **Suggested fix:** Update signature to `login(email: string, password?: string)`.
- **Developer recommendation:** Align frontend context state parameters with backend controller endpoints. ---

Defect #8: Unvalidated Negative Discounts in Finance Invoice Calculator (MEDIUM)

- **Priority:** Medium
- **Module:** Finance & Invoicing
- **Description:** Invoice creator allows setting discounts larger than subtotal, producing negative total amounts.
- **Steps to reproduce:** 1. Navigate to `/finance` module. 2. Create an invoice with Subtotal = \$1,000 and Discount = \$5,000.
- **Expected behaviour:** Form validation should constrain `discount <= subtotal`.
- **Actual behaviour:** System accepts input and records invoice total as negative (`-$4,000`).

- **Screenshot / Code reference:** [apps/api/src/controllers/index.ts:273-275](#)
- **Suggested fix:** Add validation: `if (discount > subtotal) discount = subtotal.`
- **Developer recommendation:** Enforce server-side boundary checks on invoice calculations. ---

Defect #9: Status Code Inconsistency on Invalid Token (LOW)

- **Priority:** Low
- **Module:** API Specification & Auth Middleware
- **Description:** JWT verification failure returns 403 Forbidden instead of HTTP standard 401 Unauthorized.
- **Steps to reproduce:** 1. Send request with header `Authorization: Bearer malformed_token.`
- **Expected behaviour:** Returns HTTP 401 Unauthorized (`{"error": "Invalid or expired token"}`).
- **Actual behaviour:** Returns HTTP 403 Forbidden.
- **Screenshot / Code reference:** [apps/api/src/middleware/auth.ts:22](#)
- **Suggested fix:** Update line 22 to `res.status(401).json(...).`
- **Developer recommendation:** Adhere to RFC 7235 HTTP authentication standards. ---

Test Automation Suite Deliverables (Phase 9)

The following test automation suites were authored and integrated into the codebase repository: 1. **Playwright End-to-End Suite:**

[apps/web/e2e/playwright.spec.ts](#) - TC-01: Landing Page & Branding - TC-02: Auth Flow (Login & Dashboard) - TC-03: Invalid Credentials Handling - TC-04: Projects Workspace List - TC-05: Kanban Task Board & Workflow - TC-06: CRM Lead Management - TC-07: Finance Invoicing Totals - TC-08: AI Requirements & Test Cases Studio 2. **Cypress End-to-End Smoke Suite:**

[apps/web/cypress/e2e/smoke.cy.ts](#) - CY-01: Landing Page Verification - CY-02: Session Initialization - CY-03: Negative Auth Testing - CY-04: Navigation Guard Protection - CY-05: AI Studio Assistant Rendering 3. **Backend API Automation Suite:** [apps/api/tests/api.spec.ts](#) - API-AUTH-01 through 04: JWT validation & access control tests - API-SEC-01 & 02: NoSQL injection & CastError regression tests - API-PM / CRM / FIN / AI: End-to-end module endpoint contracts ---

Final QA Scoring & Release Matrix

Audit Metric	S	G	Status / Comments
	c	r	
	o	a	
	r	d	
	e	e	
UI Quality Score	8		Modern "White & Light Green" palette. Fix text-white heading on login.
	4/	B	
	1	+	
	0		
	0		
UX Quality Score	8		Responsive layouts, intuitive sidebars, and clean navigation.
	6/	B	
	1	+	
	0		
	0		
Security Score	4	F	Critical NoSQL injection vulnerability and hardcoded JWT secret fallback.
	2/		
	1		
	0		

Audit Metric	S	G	Status / Comments
	c	r	
	o	a	
	r	d	
	e	e	
	0		
	8		
Performance Score	8/	A	Fast client bundle (<120KB) with dynamic route lazy loading.
	1		
	0		
	0		
	8		
Accessibility Score (WCAG)	2/	B	Focus indicators present; needs ARIA label cleanup on icons.
	1		
	0		
	0		
	6		
Backend API Score	8/	D	Unhandled CastErrors (500s) and missing RBAC route enforcement.
	1		
	0		
	0		
	9		
Frontend Score	0/	A	Stable React TypeScript implementation with mock fallback logic.
	1		
	0		
	0		

Audit Metric	S	G	Status / Comments
	c	r	
	o	a	
	r	d	
	e	e	
	6		
Production	2/		Requires security patches
Readiness	1	D	before production
Score	0		deployment.
	0		
	9		
Automation	5/		Full Playwright, Cypress,
Coverage	1	A	and API test suites
Score	0	+	generated.
	0		
	7		
	4.		
OVERALL	7		
QUALITY	/	C	Action Required
SCORE	1	+	
	0		
	0		

Release Recommendation

CAUTION: ### RELEASE RECOMMENDATION: **GO WITH FIXES** (MUST FIX

CRITICAL SECURITY ITEMS BEFORE PROD)

> > **Deployment Condition:** > The application MUST NOT be deployed to general production traffic until **Defect #1 (NoSQL Injection)**, **Defect #2 (JWT Secret Fallback)**, and **Defect #3 (Backend RBAC Enforcement)** are patched and verified by the automated API test suite.